The initialization method performs the following:

1. Splits the data into train and test sets.
2. Initializes and fits a `OneHotEncoder` for the categorical feature columns provided.
3. Initializes and fits a `StandardScaler` for the numerical feature columns provided.
4. Applies the `preprocess_data` method on the training and test data.
5. Defines an `evaluation` method to calculate the log loss on the X and y sets provided.
6. Invoke the `evaluation` method on preprocessed test data.
7. Defines and invokes the `_model_signature` method to easily provide the signature when logging the model.

The `predict` method calls the `preprocess_data` (*Figure 7.17)* method on the input DataFrame before performing and returning the prediction. This method is used for processing the training data and the inference data ensuring identical preprocessing for predictions.

```
@staticmethod
def preprocess_data(df, numeric_columns,fitted_scaler,cat_columns, encoder):
  if "TransactionTimestamp" in df.columns:
    try:
      df = df.drop("TransactionTimestamp",axis=1)
    except:
      df = df.drop("TransactionTimestamp")
  one_hot_encoded = encoder.transform(df[cat_columns])
  df = pd.concat([df,one_hot_encoded],axis=1).drop(columns=cat_columns)
  df["isTimeout"] = df["isTimeout"].astype('bool')
  ## scale the numeric columns with the pre-built scaler
  if len(numeric_columns):
    ndf = df[numeric_columns].copy()
    df[numeric_columns] = fitted_scaler.transform(ndf[numeric_columns])
  return df
```

Figure 7.17 – The preprocessing method for the model

As you can see in *Figure 7.17*, the fitted numeric scaler and one hot encoder are passed as input. This protects against skew between training and inference features. Notice how the Transaction timestamp is dropped. This is done because after the features from the feature tables are present, we no longer need a timestamp. The input DataFrame can be a Spark or Pandas DataFrame. This is the need for different syntax for dropping the timestamp column.